## EP6. The ellipsoid method for linear programming

Lidivine Kengne | IE 5311-001 (D01), Homework 5

**1. The main idea.** The method never reads a constraint matrix. It is handed a convex set $K \subseteq \mathbb{R}^n$ through a *separation oracle*: a routine that, for any query $z$, either certifies $z \in K$ or returns $d \neq 0$ with $d^\top x < d^\top z$ for all $x \in K$. That is the entire interface, and it is the whole point — $K$ may have exponentially many facets, none ever written down.

Write $E(z, Q) = \{x : (x - z)^\top Q^{-1}(x - z) \leq 1\}$ with $Q \succ 0$ and start from any $E_0 \supseteq K$. At step $k$, query the oracle at the centre $z_k$. If $z_k \in K$ the search ends. Otherwise the returned $d$ gives $K \subseteq E_k \cap \{x : d^\top x \leq d^\top z_k\}$, a half ellipsoid, and $E_{k+1}$ is the smallest ellipsoid containing it. With $g = Q_k d / \sqrt{d^\top Q_k d}$,

$$z_{k+1} = z_k - \frac{g}{n+1}, \qquad Q_{k+1} = \frac{n^2}{n^2-1}\Big(Q_k - \frac{2}{n+1}\, g\, g^\top\Big),$$

a rank-one update costing $O(n^2)$ arithmetic. Whatever the cut, the volume contracts by a fixed factor, $\mathrm{vol}(E_{k+1})/\mathrm{vol}(E_k) \leq e^{-1/(2(n+1))} < 1$, so if $K$ is nonempty with $\mathrm{vol}(K) \geq v$ and $\mathrm{vol}(E_0) \leq V$, a point of $K$ is found within $O(n \log(V/v))$ oracle calls. The contraction weakens as $n$ grows — each cut discards only about a $1/n$ fraction — which is exactly why the method is slow.

**2. Historical significance: linear programming is in P.** For an LP with rational data of total bit size $L$, a nonempty feasible region can be perturbed to have volume at least $2^{-O(nL)}$ and lies in a ball of radius $2^{O(L)}$. Feeding the trivial oracle "scan the rows, return a violated one" into the bound above gives $O(n^2 L)$ iterations, each polynomial. Khachiyan's 1979 proof that linear programming is solvable in polynomial time was the first of its kind and a genuine surprise: the simplex method then as now was the only algorithm anyone ran, and it is exponential in the worst case. (Klee and Minty's distorted cube forces Dantzig's rule through all $2^n$ vertices.)

**3. The lasting significance: separation equals optimization.** The deeper consequence is not about explicit LPs. Because the method touches $K$ only through the oracle, its running time depends on the number of *variables* and the cost of one separation, never on the number of constraints. Grötschel, Lovász and Schrijver turned this into an equivalence: for a well-described polyhedron, optimizing a linear function over it and separating from it are polynomial-time equivalent. An exponential family of constraints is therefore no obstacle at all, provided a violated member can be found in polynomial time. It is used far more often as a proof device than as a solver.

**4. Where this course has already used the idea.** The subtour elimination constraints of the DFJ formulation number $2^{|V|}$, and the lazy constraint callback written for Homework 1 is a separation oracle for them — the violated subtour is found by a minimum cut, never by enumeration. Homework 5 is the same idea from the other side. The Dantzig–Wolfe master holds one column per feasible pattern, so its *dual* holds one row per pattern, and the pricing subproblem — a knapsack — is precisely the separation oracle for that dual. Column generation is a cutting-plane method on the dual, and the ellipsoid method would solve the same relaxation in polynomial time from the same oracle, which is how one proves such a relaxation tractable despite exponentially many variables. The matroid polytope of EP4 is a third instance: exponentially many rank inequalities, separable by submodular minimization.

**5. Why nobody solves LPs with it.** The iteration bound is close to tight, so there is no early finish on easy instances; $Q_k$ grows ill-conditioned and needs careful high-precision arithmetic; and there is no warm start. In practice it loses to simplex and interior point by orders of magnitude. Its standing is as an existence proof and a reduction tool — the instrument that shows a problem is solvable in polynomial time, after which somebody else finds an algorithm anyone would run.